

Inkubook.com: A tale of five processes

Eric R. Willeke
Inkubook.com
Indianapolis, USA
eric.willeke@gmail.com

Abstract

Inkubook.com came into existence in March 2008 when an existing software development and marketing organization received a new CEO who immediately tasked the team with building an entirely different product. This paper describes the sources of pain that led to an evolution from the existing Scrum process through four major changes as the team's process shifted to meet the team's goals and management's demands. Then, the ramifications and further changes of the resulting kanban approach are explored.

1. Who is Inkubook?

Inkubook is an emerging player in the online photobook industry. Using the inkubook.com web site, users are able to upload their images, utilize those images in the creation of a highly personalized book, and order high-quality printed copies of those books. The entire process from initial contact through ordering the printed book is entirely browser-based.

Inkubook is owned by ASI, Inc. which also owns AuthorHouse, iUniverse, Xlibris, and WordClay. ASI is the world leader in the self-publishing industry, providing a publishing platform with a wide variety of options for getting authors' work into publication.

The team built the core software using Microsoft technologies, including Silverlight, WCF, ASP.NET, and SQL Server. It additionally leverages Amazon's cloud storage offerings for providing fast, high-availability access to the user's assets.

1.1. Business context

Inkubook lives in a different world from the rest of the ASI companies. Although it shares office space, information technology, human resources, and accounting services with the other ASI businesses, Inkubook remains remarkably isolated at a cultural level. The company maintains its own development infrastructure, with only core services being routed through the corporate IT group. Only recently has the Inkubook development team interacted significantly

with the rest of the corporation's development staff. The team's practices and business processes are very isolated from the workflows used by the rest of ASI. These factors have allowed Inkubook the opportunity to display an incredible level of business and process agility throughout the product's life.

1.2. The Inkubook journey overview

The team that became the Inkubook team assembled between March and August of 2007. The team was created as part of a new technology initiative drive manuscripts into the various corporate offerings for fulfillment. Development on this product continued through the end of January 2008. However, the board of directors hired a new CEO to take over all of the ASI companies in late 2007. In January the new CEO decided to abandon the existing project and enter a new market. Thus, on the first day of February 2008, Inkubook.com was born.

At conception, the product consisted of a slide deck containing the Director of Information Technology's vision of the product, a full team that had already passed through the forming, storming, and norming steps on the Falcon project, and an established development infrastructure. The CEO decreed that the initial product should be ready for release by the end of March, giving the team sixty calendar days to create a new product and prepare it for initial release. These constraints drove many of the decisions the team made during that period.

Despite selection of the very new Microsoft Silverlight as the user experience front-end toolset, the team hit the deadline with an acceptable set of features in the closing days of March. The development and releases of Inkubook fell into a consistent rhythm, interrupted periodically by Microsoft's updates to its Silverlight platform. Finally, in October 2008, Microsoft released its final 2.0 of the Silverlight platform and Inkubook released a compatible, final version three days later. Inkubook has continued to release new features on a weekly basis as it focused

and adapted its software to meet the growing understanding of the photobook business.

2. The five processes

The vast changes in business focus, team size, and market understanding have provided a constant stress on the ability of any development methodology to cope with the needs of the business. Luckily, the Inkubook team was not constrained by corporate guidelines, an oversized team, or tradition-bound leaders. As a result, the team was able to shift its approach to developing software rapidly, addressing the specific risks and constraints of each phase of the products' evolutions.

The initial development effort used nearly textbook Scrum. However, when the mandate arrived that a product would be delivered in sixty days, the team shifted to a methodology best described as "burn-out inducing chaos". After the initial release, the technical leaders of the team insisted that the schedule slow down to avoid burnout and avoid staff turnover. As a result, the process moved back to "Scrum", but it was really a flow-based, iterationless version of Scrum. After three months of being in the same "sprint", the team recognized that flow was working well for getting things done and abandoned estimates, implicitly organizing around a work in progress (WIP) limit of two MMF's. Finally, after several more months passed, the team accepted that they were using a work-limited, pull-based Kanban approach and updated the usage and terminology to reflect that fact. The use of a kanban implementation survived a round of layoffs, an extreme change in team and management composition, and is still being used today.

The following sections elaborate on the nature of each of these process variants.

2.1. Scrum

The team's leaders had practiced Scrum and were experienced with its approaches and intents at the start of the Falcon project. Despite the lack of a Certified Scrum Master (CSM), the team was able to implement Scrum with a reasonable degree of success during the Falcon portion of the team's life. The team of fifteen divided into two equal sub-teams. The sub-teams each consisted of an analyst, a graphic designer, two developers, two testers, and one architect. Each sprint the teams would individually commit to some amount of functionality and attempt to finish it before the end of the sprint.

The basic approach to Scrum changed a number of times during these first eight months. For a long period of time we struggled with getting the requirements accurate, so we moved the analysts outside of the

scrum structure and fed in completed screen compositions rather than story-level requirements. This forced the marketing group to commit to specifics, because they desired great control over the results. If the details were minor, the changes were not a problem. Unfortunately, the changes frequently reflected fundamental changes or misunderstandings with the nature of the work. As a result, the team's ability to work side-by-side with marketing degraded over these first few sprints to where any changes were confrontational rather than collaborative. These experiences caused more reworking of the timings and relationships between roles, but none of the versions ever felt quite right with our team's highly specialized roles.

Unfortunately, things didn't feel quite right during this period. There an ongoing sense of pain arose within the development team. People's job enjoyment fell because of the various small frictions within the process and between the teams. This friction grew fastest between the business analysts and the marketing group. The marketing team knew what they wanted, and desired to be heavily involved in the development of the product they would have to market. On the other hand, the designers and analysts felt it was their responsibility to create and design the product, leading to resentment as each group tried to achieve their vision for the product. This lack of shared vision led to a continual back and forth between the two teams. The resulting "design by committee" effect resulted in a product inferior to either initial design.

Another source of stress arose as the team struggled to find the proper balance of documentation for new features. The marketing group desired to see high resolution visuals of what the screens would look like. The developers wanted simple wireframe screen flows and enough of a design guide that they could work out the proper HTML without having to count pixels in a PNG file. The testers wanted complete use cases and behavior response lists. This contributed to a lot of useless discussions and the perceived need for more people writing the documentation. Fortunately, the team didn't fall too far into this trap, and managed to work out a marginally acceptable requirements document that everybody could use to work. Unfortunately, it would be many months before the team found a way that greatly decreased the costs associated with documentation.

On the good side, the structure of Scrum was easy to manage, and the results were consistent and predictable. Everybody did their best to help other roles throughout the process to keep work moving, and the team gelled well within each role. Additionally, the leaders among the developers and architects learned to work very effectively with marketing, creating the

desired product despite the challenges in defining the goals and vision.

2.2. Ad hoc, called “kanban”

As 2007 drew to a close, and especially through January of 2008, the team leadership began seriously considering a change away from Scrum. As discussed previously, management effectively cancelled the Falcon project and assigned the team a new product on February 1, 2008. Along with this new project, they dictated that the product would be ready to release with minimal functionality on March 31, 2008. This threw the team into major state of panic. The deadline of 60 calendar days also ruled out our existing version of Scrum; the team believed it would need an order of magnitude better lead-time than the existing 30 day iterations could provide.

The team moved seamlessly into pure chaos. The four architects looked at the presentation deck describing the concept, sketched out a data model, a screen flow, a technology list, and split it into a few high-level areas of focus. The developers picked out areas they wanted to focus on, and then grabbed whatever bit of work they could handle next and dove into it. In a strange way, the team divided into silos in ways counter to expectations. People did not talk or collaborate within their role; instead, marketing would often be the only unifying force as each architect and developer dug into their own vertical slice of the application. The test team would take builds as they came and find things to test, providing guidance and feedback on what felt right and what was clunky and buggy. The team still gathered for 10 minutes every morning, but conversation would stop after that. This pattern continued, with massive overtime, for the next two months.

Remarkably, this was the most productive two months of any team that I worked with in my career. The team not only hit its deadline, it managed to include a few extra features than were expected in the first release and do so at a reasonably high level of quality. The product was not ready for the market, but it was ready for the critical decision to continue funding it, and restored a great deal of confidence in the team among the management layers.

Unfortunately, the cost of this effort was a great deal of burnout within the team. The pace for those sixty days was extremely demanding, and the emotional makeup of the team would not allow this behavior to continue long term. Team members' families were constantly angry with long hours and emotional frustration with the work to be done. Both of the architects conveyed their intent to leave if this were to continue. The analyst group continued to feel

increasing friction with the rest of the team because they had been relegated to the role of note takers and history writers.

Worse, there was no way for the team's leadership to understand the team's progress. Each developer was able to explain the status of the current task, but there was no high level view of what tasks remained, and nobody took the time to build that task list. The team existed in a perpetual state of done vs. not done with no incremental reporting available.

If this had been limited to sixty days, I think the team would have been fine with the situation. However, we spent another four months in a state of perpetually “almost ready to release”. Oddly, it was the marketing team making this decision, determining that the market would not accept the product in the current form and asking for another handful of key features. Each time, everybody agreed it was the right decision, and the team moved into another wave of pushing features out as quickly as possible. Use of the task board for visual control waned and slowly died as the team decided that storing the task info in Team Foundation was just as useful. The chaos continued, and both accountability and manageability suffered greatly.

2.3. Basic flow, called “Scrum”

The team managed to push the initial release to production, along with a number of follow-on releases containing additional features. The team was more than ready for a change, and the marketing team was ready to start marketing instead of helping build a product. There were still many more features to build, but the application could start earning its own keep. The team's work mix at this point was half new features and half marketing support. The marketing support items were things that would directly assist with imminent marketing campaigns, reporting, seasonal HTML redesign and other low-effort, high payoff tasks.

The features tended towards longer, strategic features aimed at countering the specific strengths of various competitors. As a result, the team started doing sprint-based development again, intending to return to our previous Scrum implementation. The analysts had the first major feature set ready for development, marketing was on board, and the technical team had already provided the capability feedback. The team estimated the work, shared it at sprint kickoff, and realized there was only enough work to fill half the sprint! The analysts went back to start the feature set after that and development went off to build the well designed feature. About the time the analysts completed the next feature set's requirements, the

BA Team (2)		Tech Team (2)						SQA Team (2)			
Queue	In Progress	Ready to implement	Req. in progress	Task queue	In Progress (4)	Completed	Ready for testing	In Progress	Ready to deploy	In Production	
Queue		In Progress				Completed					

Figure 1 - The Inkubook kanban board

developers were ready to receive it. This continued for a month or two, and production levels were high.

However, what the team did definitely was not Scrum. First, iterations were forgotten, and the team did not do any kickoffs or wrap-ups. The sprint titled “Sprint 15” went on for two and a half months, defining features, adding to the backlog, working, and releasing without any attention to the boundaries. The team estimated each feature, and management held the team to those estimates, but the overall feeling and efficiency was much higher than it was in our original Scrum implementation.

The team's morale was growing higher, the team increasingly gelled more effectively across all roles, and the software continued releasing at a rate that kept management happy with the progress. The value stream did not change, but nobody coerced it into an arbitrary release framework. Management found it easier to track each group's developments, and marketing found that their lead-time reduced significantly.

This carried costs to the team, though. The team still struggled with some individual tasks taking much longer than they ought to. Testing suffered trying to plan around the random arrival patterns for different features. Worse, the seemingly random selection of features to implement left the developers feeling distracted and aimless, because they could not see the future path of the product. The business analysts sat on an ever growing pile of completed requirements, with the expected effect that many of them were no longer entirely accurate by the time they were ready to implement. As the analysts stopped feeling pressure to have more features prepared immediately, they began

spending more time discussing the ideal form of the features. Unfortunately, this focus led to analysis paralysis on the front end, and the resulting meetings consumed a lot of marketing and architect time that could have been spent focused on improving the product.

2.4. Advanced flow, implicit WIP

The team continued working with a similar approach into autumn 2008. However, two important changes occurred that caused a distinct change in the feel of the process. First, management stopped asking for specific estimates on most tasks after the team released calendars in time for the U.S. Thanksgiving holiday. Second, the team settled into an implicit limiting of features to two at a time. Through trial and error, the team arrived at an understanding that “more isn’t better”, and that limiting how many tasks were floating around at once helped all levels within the team focus on what needed done.

Nothing else really changed, but overall output went up again, people tended to be more pleased with their output, and marketing's lead-time reduced again. For most features, the total lead-time from request to release was 20-25 days, unless a higher priority feature pushed a feature out. Development abandoned or deferred very few features once they made it into engineering. The only oversized queue continued to be the set of features with requirements written and screens drawn, but without any engineering effort. The stale feature requirements continued occurring.

Marketing worked very well with many of the roles. Unfortunately, the friction between marketing

and analysts grew worse by the week. The screen designers were angry and felt diminished by marketing's insistence of a great deal of control over the process. Marketing doubted the ability and focus of the designers, and grew incredibly frustrated when every minor feature request seemed to involve a complete rework of the interface. Neither group was effectively looking forward at the needs of future expansion, which led to a lot of unneeded rework that could have been reduced. The results of this frustration began to spill over into other groups as marketing began ignoring the requirements and working through the actual desired implementation with the developers, leaving the testers to figure out what to test. The test team was used to working from very detailed requirements, but was no longer receiving them, and had to find a better way to work. This led to the analysts spending more time updating documents for work that had been effectively completed. It also increased the visibility of the costs incurred by the test team's insistence on receiving fully details documentation. The team was realizing it really cared about validation, but cared surprisingly little about verification.

2.5. Full kanban with WIP limits.

Finally, in early 2009, the team's leadership decided to embrace fully the approaches known as kanban software development. The team chose to enforce a hard limit on work in progress (WIP) for each station, determined that each individual and group would be free from having more work assigned before they were finished with the previous work, and planned that the team would attempt to swarm on any given feature as much as possible. As a result, the WIP limit was set to two for each stage, with an overall WIP limit of six. However, the team rarely reaches this overall limit, as the test group usually only has one feature in their area. The team also introduced a class of service known as "the easy button" to allow short, 1-2 day tasks to slip past the bigger ones when there was a momentary blockage or inefficiency around one of the larger tasks. **Error! Reference source not found.** demonstrates the physical board's layout.

People were initially very skeptical, but the overall feeling about the system was high as the weeks went on and the team's productivity continued to rise. The lead-time on specific feature sets settled to 10-14 days for average feature sets, with some coming in much lower. The biggest deviation to this was the one time when we allowed our WIP to rise to three major features in development, causing a bit of a train wreck at integration time. Since this pain, with its week of lost time integrating, occurred less than a month after

switching into kanban, it provided a strong and early reinforcement of the value of limiting WIP. Marketing continues to gain value from the "easy button", allowing the team to respond with great agility to specific customer needs and market opportunities.

The biggest issue the team experiences with kanban is that much of the clean up and refactoring work that used to happen in the down times isn't being done as consistently. As a result, the team maintains a backlog of these items and matches them with new work that occurs in the same area. This limits marketing to paying for previous technical debt only when they request features affecting that area of the architecture. As a result, the team can somewhat quantify the cost to maintenance caused by earlier shortcuts. The list also serves to remind the team of areas demanding additional care and prevents compounding of the debt. Unfortunately, friction remained between marketing and the analysts over the details of the screens.

The development team became incredibly confident in their ability to deliver the right value quickly, and began to enjoy their job significantly more as a result. The features began to be selected based on user and partner requests rather than arbitrary selection, leading to a powerful emotional feedback loop between the developers and marketing. The response time for most requests dropped to where marketing never felt like they were having to wait unduly long for any feature. Best yet, the architects were able to spend part of their time working with other groups in the parent company, which would lead to interesting effects in upcoming months.

2.6. Kanban stress test

Two months into the kanban incarnation of our process, the team suffered layoffs of 60% across all roles as the executives sought to bring costs in line with revenue expectations. The cuts left a team of 26 people with only 10 people, and 20% time of 2 other individuals. After the layoffs, the team's composition was two marketing team members, two developers, one tester, one graphic designer, one graphic asset specialist, one architect, and a development manager who also spends about 50% time writing code. The flow slowed, but did not stop. In hindsight, the results were easily predictable: double the remaining SLA allowance on all WIP, and then reduce the WIP limit by half to maintain the existing SLA.

With the reduced team size, the Inkubook team found that the visual control is less important than it was previously. The entire team sits within easy conversational distance, and everybody is hyper-aware of the current marketing and development goals

through the daily stand up meetings. As a result, the discipline on the board is slipping, and the team is considering a test run of the board not existing at all. The team may discover the visual control is just a crutch that for smaller teams, especially since Team Foundation duplicates the information with much higher fidelity.

Many other useful results emerged. The decreased number of communications paths caused the overall team to remain nearly as productive as before the layoffs. The removal of most of the analyst team instantly removed the friction with marketing. The removal of most of the test team left a single member who was incredibly adept at working with whatever inputs she was given to provide meaningful test results. The remaining tester and designer began reporting directly to the development lead, who had been a senior developer before the layoffs. The smaller team also changed the tone and feeling of the morning meeting to more of a fireside chat than a status report or formal meeting, yet still allowed all individuals (including marketing) to have a part within a ten minute session. The remaining architect only had two developers to support, and was able to provide a lot of directed design training and help share several powerful thinking tools for understanding the current and future architecture of the system. This had the effect of further freeing the architect to do more active development work and help other teams more frequently. The developers spent more of their time working as a pair, and there were very few challenges they couldn't handle with this arrangement.

3. Kanban goes to Bloomington (Indiana)

Other groups within AuthorHouse adopted a very similar kanban process as individuals previously assigned to Inkubook assumed leadership positions within their teams. These changes were initially very much imposed by management, but the impact was reduced by the seeding of individuals across several levels, including the CIO, architecture lead, a software architect working within the team, and a data architect. This allowed the changes to be led by example within the team rather than feeling completely imposed. Each of the roles learned from a member of their own team, and the outside stakeholders were kept at bay by a rather powerful new CIO.

The first new team to adopt the kanban process found a great deal of immediate benefit from the raised visibility available via the kanban board, which allowed the many stakeholders to see what their individual requests had to compete against. The visibility around the larger corporate initiatives allowed the team to justify periods of ignoring other

stakeholders, but also permitted a very clear and open discussion around priorities and the occasional introduction of other work to support opportunities in the market.

This degree of flexibility led quickly to a portfolio based approach for allocation of development time between the various initiatives and maintenance efforts. This allows the team to make the decision to address cost saving measures for recurring development processes, freeing the developers to address new features after that initial investment is complete. Prior to adopting the process, their 10 developers spread themselves across 10 different projects. Now, the entire team of developers is focused onto two or three projects.

The process tends to change every two or three days in some minor way, always pushing to eliminate waste within the process and focus the team on the efficient delivery of the right value. This led to organizational changes, with the database experts being pulled out to their own sub-team with their own class of service for meeting the reporting needs of the organization, saving the development team from the distraction of the steady stream of requests. Worldwide application development has been pulled into a single distributed functional group, allowing requests to go through a single routing path and be compared appropriately with other outstanding requests. This has steadily improved both the overall release rate of new features and the overall capability level of the development staff.

4. Afterword

Both Inkubook and the core AuthorHouse team found a great deal of value in the explicit recognition that their processes are indeed an asset to the development team. Acknowledging the ability to change the process to fit the current context led to a great deal of value and cost savings as the different teams adapted quickly to achieve their goals.

It is important to recognize that none of the processes that the teams used were inherently bad; instead, they were contextually appropriate until the context changed. These lessons are key when considering decisions about how much change to take on, and how often to accept that change.